

LIFEMOTE MEDIATEK-BASED CPE AGENT FIRMWARE SPECIFICATION

1 Scope

This document contains the specification for the requirements of the Lifemote data gathering agent on MediaTek-based CPEs. The requirements are grouped as those relating to Deployment and Commands. All requirements are to be considered mandatory, and no part shall remain unfulfilled without losing Lifemote functionality.

2 Deployment

The Lifemote data gathering agent shall be downloaded by the CPE via HTTPS each time the CPE boots, and if for whatever reason the agent is terminated during runtime. The software module implementing this functionality is called the Lifemote Deployer – details are presented and source code linked below.

2.1 TR-069 Options

Lifemote agent shall be toggled on and off via a TR-069 option. The TR-069 options supported by the CPE shall be as follows:

- **Device.X_LIFEMOTE_EXT.LifemoteAgent.Enable:** True or False, Read/Write. When set to True, will immediately activate the Lifemote Deployer specified in Sec. 2.2, and also add the Lifemote Deployer launch to the boot script of the CPE, such that the Lifemote Deployer will be launched as a background process each time the CPE boots. When set to False, will immediately terminate the Lifemote Deployer background process using the SIGTERM signal ¹, and remove the Lifemote Deployer launch from the boot script of the CPE such that the Lifemote Deployer will not be launched the next time the CPE reboots. Note that toggling this option from True to False and back to True should prompt an update of the Lifemote agent.
- **Device.X_LIFEMOTE_EXT.LifemoteAgent.URL:** String, Read/Write. Specifies the URL from which the Lifemote agent will be downloaded by the Lifemote Deployer. This string shall be recorded as an environment variable named `LIFEMOTE_AGENT_URL` in a manner accessible by the Lifemote Deployer. When this field is set, the Lifemote Deployer shall be automatically terminated using the SIGTERM signal and restarted. This behavior shall allow a new Lifemote Agent to be deployed via a single TR-069 command.
- **Device.X_LIFEMOTE_EXT.LifemoteAgent.State:** True or False, Read only. The CPE shall set this parameter to True if the Lifemote Deployer background process is running, and to False otherwise.

¹Note that the background process shall be terminated with SIGTERM and not SIGKILL so that the Lifemote Deployer can run cleanup routines upon intercepting the SIGTERM signal.

2.2 Lifemote Deployer

The Lifemote Deployer is a small software module that shall be run as a background process. This module periodically checks for a running Lifemote agent on the CPE, and when it finds the agent to be absent will download the agent via HTTPS GET of the `LIFEMOTE_AGENT_URL` provided via TR-069 and run it as a background process.

If the Lifemote Deployer cannot download the agent via HTTPS GET, it shall keep retrying at increasing intervals. The first interval shall be uniformly distributed between 5-10 minutes, then 10-20 minutes, then 20-40 minutes, then 40-80 minutes, and finally 60-120 minutes. The retries shall remain at every 60-120 minutes after this point until the agent is successfully downloaded. The randomization for this functionality is meant to spread out the loading of the server offering the agent.

If `LIFEMOTE_AGENT_URL` is empty or undefined, the Lifemote Deployer shall terminate immediately upon launch. This behavior shall be visible via the `Device.X_LIFEMOTE_EXT.LifemoteAgent.State` TR-069 parameter. Note that `LIFEMOTE_AGENT_URL` can be modified simply by the `Device.X_LIFEMOTE_EXT.LifemoteAgent.URL` without setting `Device.X_LIFEMOTE_EXT.LifemoteAgent.Enable`.

Sample BASH code for the Lifemote Deployer can be downloaded from Lifemote². The CPE vendor is encouraged to use this code directly, although any code with exact equivalent functionality is acceptable.

3 Commands

The Lifemote agent shall exercise the following commands during its operation – functionality of these shall be tested by the CPE vendor.

²<http://dream.demo.lifemote.com/v1/scripts?type=phoenix>

General Functionality

Functionality	Command	Notes
Shell scripting environment		Should support conditionals, parameter substitution, slicing, loops functions, arithmetic expressions
Send data using HTTPS	curl (HTTPS GET and POST)	Should support plain HTTP requests and HTTPS requests. The CPE vendor shall provide any necessary embedded certificate chains for the CPE to execute HTTPS GET and POST commands.
Set of basic commands and tools to perform text output and manipulation	echo, cat, tr, grep, egrep, cut, head, tail, date, sleep, ps, kill, read, hexdump	echo command should support printing without appending new line, i.e. echo should support "-n" option
Retrieve network interface information and related counters belonging to network interface	ifconfig	
Display DHCP leased IPs	dnsmasq, udhcp or equivalent mechanism to view DHCP leases	Leased IPs should be reliably stored in a file in the filesystem (e.g. /var/dnsmasq/dnsmasq.leases or var/udhcpd/udhcpd.leases). In case of udhcp as DHCP service, hexdump with '-v' and '-e' option support
Display bridge information	brctl show	Should be able to list bridges in the network stack
	brctl showmacs	Should be able to show MAC table of the bridge which includes learnt MACs, aging counters and port information
	brctl showstp	Should be able to show STP information
Obtain firmware version through command line		
Display ARP table	arp	
Support for SSL	openssl	
Show various information like ARP, route	ip route or route, arp or /proc/net/arp	
To be able to obtain DSL line stats (for DSL gateways)	/proc/tc3162/*	Should be able to show DSL power, SNR, attenuation, current rate, attainable rate for both uplink and downlink

WiFi Functionality

Functionality	Command	Notes
List stations associated to specific Wifi interface and display information about associated Wifi stations	A wrapper CLI utility around stainfo and stacountinfo IOCTLs (i.e. iwpriv IF show stainfo, iwpriv IF show stacountinfo)	• be able to print to stdout directly instead of kernel ring buffer • include usage counters (total transmitted bytes, total received bytes) • include PHY rate of both last transmitted and received packet • include maximum possible MCS and Rate • include an "in network" timer which specifies association duration elapsed since the time when the station associated with the Wifi interface • include RSSI information
Display Wifi driver counters for a given interface	iwpriv INTERFACE stat or a wrapper CLI utility	Should include counters such as total transmitted frames, total retransmissions, total failed frames
Disable MIB bucket	iwpriv INTERFACE set MibBucket=0	
Display CCA information	iwpriv INTERFACE mac MAC_REGISTER	Given register can be a counter that is cleared on read and provides channel busy time between two reads
Display WiFi noise level for current channel		
Display air time information and interference counters about current Wifi channel		Counters like Tx duration, In BSS airtime, OBSS airtime, Non-Wifi Airtime
Perform channel scan to discover neighbor APs	iwpriv INTERFACE set SiteSurvey=1 and iwpriv INTERFACE get_site_survey	Scan info should include neighbor SSID, channel, channel width, supported Wifi standard (e.g. b/g/n/ac)
Obtain current status of a Wifi interface	iwconfig	Should include current SSID, current channel number, current channel width, current channel spec